

You've got three containers to run.

So you open a terminal and start typing.

```
docker run -d --name redis redis:alpine — fine. docker run  
-d --name web -p 8080:80 --network app-net nginx:alpine  
— okay.
```

Now add a database, a background worker, and a metrics exporter. Suddenly you have a 200-line shell script that breaks every time someone joins the team and forgets to run it in the right order.

There's a better way.

What is Docker Compose?

Docker Compose is a tool for defining and running multi-container applications using a single YAML file.

Instead of juggling `docker run` commands, you describe your entire stack — services, networks, volumes — in one place.

```
docker compose up
```

Starts everything. `docker compose down` tears it all down.

That's the whole mental model.

Getting Docker Compose

Ships with **Docker Desktop** on Mac and Windows — nothing extra to install.

On Linux, install it as a plugin:



```
1 # As the Docker CLI plugin (recommended)
2 apt install docker-compose-plugin
3
4 # Or grab the standalone binary from GitHub releases
```

From docker run to Compose

Two services, zero configuration headaches:

```
1 services:
2   web:
3     image: nginx:alpine
4     # Expose nginx on port 8080 on your host
5     ports:
6       - "8080:80"
7
8   cache:
9     image: redis:alpine
10    # Redis is only reachable by other services in this compose project – not exposed to the host
```

Both services land on the same internal network automatically — no `--link` flags, no manual wiring.

vs. Raw Docker Commands

The equivalent commands you'd otherwise need:



```
1 docker network create my-app
2 docker run -d --name web --network my-app -p 8080:80 nginx:alpine
3 docker run -d --name cache --network my-app redis:alpine
```

Same result — but now it's a script, not a file. No version control, no reproducibility.

Why This Matters Beyond Convenience

The Compose file is **checked into version control**. Your entire local development environment is code.

It evolves with your project, gets reviewed in pull requests, and is **consistent across every developer's machine**.

When a teammate clones your repo, they don't need to know any of this. They just run `docker compose up`.

It also maps directly to **Docker Swarm**, and gives you the mental model that scales to **Kubernetes**.

Key Takeaways

- Replaces ad-hoc `docker run` chains with a single declarative YAML file
- `docker compose up` starts all services; `docker compose down` removes them cleanly
- Services are automatically placed on a shared internal network — no manual wiring required
- The Compose file lives in version control — your dev environment is reproducible for the whole team
- Docker Desktop bundles Compose; Linux users install the `docker-compose-plugin` package

Try It Yourself

The full example is in the repo at `examples/01-intro/`.

Clone it and run `docker compose up` — nginx will be live at **`http://localhost:8080`** in seconds.

How do you currently manage multi-container apps locally? Shell scripts, a Makefile, something else?